

MeetVideoCapture: technical solution

The application captures already-rendered Google Meet or Zoom Desktop video from a top-level X11 window, displays the selected crop in a Qt/QML preview, and records it to MKV/H.264 on demand.

Qt 6 / C++ / QML

X11 window capture

GStreamer

No browser / Zoom hacking

What the application does

- Finds a top-level Meet, Zoom, browser, or source application window through X11.
- Captures pixels from that window by `XID`.
- Crops the meeting-video region inside the selected source window.
- Shows only the crop and, when requested, writes it to a recording file.

What the application intentionally does not do

- It does not intercept an internal `WebRTC MediaStream`.
- It does not extract a `VideoTrack` from the browser.
- It does not use tab capture APIs or Chrome extensions.
- It does not use the Zoom SDK to extract a media stream.
- It does not use Selenium, CDP injection, packet sniffing, or similar hooks.

1. Core idea

Google Meet in a browser and Zoom Desktop already decode video and render it into a top-level application window. The backend therefore works below the application layer: it captures the already-rendered X11 window image. For Meet this is not a WebRTC stream, and for Zoom this is not Zoom's internal media stream.

```
Google Meet in a browser or Zoom Desktop
->
source app decodes and renders video
->
X11 capture of the top-level source window by XID
->
crop meeting-video region relative to the source window
->
Qt/QML latest-frame preview
->
optional MKV/H.264 recording
```

2. Window capture

`X11WindowFinder` reads the list of top-level windows and collects title, class name, XID, geometry, PID, and source profile. Zoom meeting windows and browser meeting windows are sorted above other windows, but any valid top-level window can be selected. After selection, the controller passes the XID and geometry into capture settings.

```
ximagesrc xid=<sourceWindowId> use-damage=false show-pointer=false ! video/x-raw,framerate=30/1 ! videocrop left=<l> top=<t> right=<r> bottom=<b> ! videoconvert ! video/x-raw,format=BGRA ! queue leaky=downstream max-size-buffers=1 ! appsink emit-signals=true sync=false max-buffers=1 drop=true
```

The UI stores the crop as `x/y/width/height`. GStreamer `videocrop` accepts `left/top/right/bottom`, so the controller converts the crop using the selected source-window dimensions:

```
left = cropX top = cropY right = sourceWidth - cropX - cropWidth bottom = sourceHeight - cropY - cropHeight
```

3. Preview without latency accumulation

Preview uses a latest-frame strategy: only the newest frame is stored. If the UI has not rendered an older frame yet, the newer frame replaces it. There is no unbounded preview queue, so obsolete frames do not accumulate.

- `appsink drop=true`, `max-buffers=1`, `sync=false`.
- `queue leaky=downstream`, `max-size-buffers=1`.
- `VideoFrameProvider` stores the latest `QImage` and coalesces update signals.
- `VideoPreviewItem` renders the frame through a Qt Quick scene graph texture node.

4. Recording

Recording does not block the UI and does not slow down preview. While recording is active, capture frames are fed into `VideoRecorder`; when recording is off, that feed is disconnected. The recorder has a bounded queue and drops frames under overload instead of increasing latency.

```
appsrc is-live=true format=time ! queue max-size-buffers=30 leaky=downstream ! videoconvert !
video/x-raw,format=I420 ! x264enc tune=zerolatency speed-preset=ultrafast key-int-max=30
bitrate=2500 ! h264parse ! matroskamux ! filesink location=<source-profile-recording-
YYYYMMDD-HHMMSS.mkv>
```

The default format is MKV/H.264. MKV is used because it is more robust for live recording: after an abnormal process exit, the risk of producing a completely unusable file is lower than with MP4.

5. Threads

UI thread

QML, controller state, recording timer, and scene graph scheduling. Long X11/GStreamer commands do not run here.

Window refresh worker

`X11WindowFinder` enumerates top-level windows in a separate `QThread` and returns a ready list to the UI.

Capture command worker

Start, stop, restart, and crop-update commands run serially in a separate `QThread`, without blocking QML.

Capture streaming path

GStreamer streaming threads, appsink callback, FPS stats, black/stale sampling, latest frame update.

Recorder worker

appsrc feeding, encoder, muxer, file sink, EOS, and finalization without waiting in the UI thread.

Bounded data flow

There are no unbounded queues. Preview does not wait for the recorder. Under overload, dropping is preferred over latency growth.

6. Fallback

If XID capture returns black or stale frames, the application can switch to `ScreenRegionFallback`. In that mode `ximagesrc` captures the visible screen rectangle described by the crop coordinates. This is not tab capture and

not an extension: it is desktop pixel capture.

Fallback limitation: if another window covers the selected video region, that window will also be captured.

7. MVP limitations

- Ubuntu X11 only. A Wayland/PipeWire portal backend can be added later.
- A minimized or hidden Meet, Zoom, or source application window may stop rendering.
- XID capture depends on the compositor, GPU, and source application, and may sometimes be black or stale.
- Crop is set numerically relative to the source window; a visual overlay can be added later.

8. Future Windows and macOS expansion

The Linux/X11 backend should remain the first implementation of a shared architecture, not a separate Linux-only branch. To support Windows and macOS later, platform capture must stay separated from common frame processing.

```
Platform capture backend
->
common VideoFrame / frame abstraction
->
crop / timestamp / format normalization
->
common frame pipeline
->
preview / analysis / recorder
```

Platform-specific parts are window enumeration, native source-id selection, permission model, the actual capture API, and native pixel-buffer or texture handling. Common parts are the frame abstraction, crop settings, timestamps, FPS/statistics, black/stale detection, bounded/latest-frame delivery, recorder interface, file naming, and a shared warnings/errors model.

Linux

Current X11 path: XID -> GStreamer ximagesrc -> crop -> common frame. For Wayland, the natural extension is PipeWire / xdg-desktop-portal -> selected window or display stream -> common frame.

Windows

Main path: HWND / selected window -> Windows Graphics Capture -> Direct3D texture frames -> crop/convert -> common frame. The backend should not be based on GDI screenshots or a screenshot loop.

macOS

Main path: SCWindow or display -> ScreenCaptureKit -> CMSampleBuffer / CVPixelBuffer -> crop/convert -> common frame. The implementation will partly use Objective-C++.

Recorder

The first cross-platform step can keep the shared GStreamer recorder: appsrc -> x264enc -> matroskamux -> MKV/H.264, provided that the GStreamer runtime is shipped with the application. Later native recorder backends are possible: Linux -> GStreamer, Windows -> Media Foundation / hardware encoder, macOS -> AVAssetWriter / VideoToolbox.

Windows limitations: user consent or a picker may be required, protected windows may not be capturable, minimized windows may stop updating, and GPU/driver/compositor behavior can affect the result. In the future it is preferable to avoid unnecessary GPU-to-CPU copies if preview and recorder paths can consume textures directly.

macOS limitations: Screen Recording permission is required, the user may deny access, protected content may not be capturable, and minimized or hidden windows may stop updating.

Main rule: do not mix the platform capture backend with common processing and recording logic. Linux/X11, Windows, and macOS should be different implementations of one capture interface. Then the Ubuntu X11 MVP remains the first platform implementation of a shared meeting/window capture architecture, not a Linux-only dead end.

Summary: MeetVideoCapture does not hack the browser, does not touch WebRTC, and does not extract Zoom's internal media stream. It captures what the source application has already rendered into an X11 window, crops the required region, and sends it to a low-latency preview and recorder.